

API Documentation

Enhanced Agentic AI SDLC System API

Base URL: `http://localhost:8000`

Table of Contents

- [Authentication](#)
- [Projects](#)
- [Tasks](#)
- [Documents](#)
- [Agents](#)
- [Health & Status](#)
- [Error Handling](#)

Authentication

Currently, the API operates without authentication for development purposes. In production, JWT-based authentication will be implemented.

Future Authentication Headers

```
Authorization: Bearer <jwt_token>  
Content-Type: application/json
```

Projects

Create Project

POST `/projects/`

Create a new project in the system.

Request Body:

```
{  
  "name": "string",  
  "description": "string (optional)"  
}
```

Response:

```
{
  "id": 1,
  "name": "My Project",
  "description": "Project description",
  "status": "active",
  "created_at": "2024-01-01T00:00:00",
  "updated_at": "2024-01-01T00:00:00"
}
```

Example:

```
curl -X POST "http://localhost:8000/projects/" \
  -H "Content-Type: application/json" \
  -d '{"name": "AI SDLC Project", "description": "Enhanced development lifecycle"}'
```

Get All Projects**GET** /projects/

Retrieve all projects in the system.

Response:

```
[
  {
    "id": 1,
    "name": "Project 1",
    "description": "Description 1",
    "status": "active",
    "created_at": "2024-01-01T00:00:00",
    "updated_at": "2024-01-01T00:00:00"
  }
]
```

Get Project by ID**GET** /projects/{project_id}

Retrieve a specific project by its ID.

Parameters:- `project_id` (integer): The ID of the project**Response:**

```
{
  "id": 1,
  "name": "Project Name",
  "description": "Project description",
  "status": "active",
  "created_at": "2024-01-01T00:00:00",
  "updated_at": "2024-01-01T00:00:00"
}
```

Tasks

Create Task

POST /tasks/

Create a new task within a project.

Request Body:

```
{
  "title": "string",
  "description": "string (optional)",
  "project_id": "integer",
  "priority": "string (optional, default: medium)"
}
```

Response:

```
{
  "id": 1,
  "title": "Task Title",
  "description": "Task description",
  "status": "pending",
  "priority": "medium",
  "project_id": 1,
  "assigned_to": null,
  "created_at": "2024-01-01T00:00:00",
  "updated_at": "2024-01-01T00:00:00"
}
```

Example:

```
curl -X POST "http://localhost:8000/tasks/" \
  -H "Content-Type: application/json" \
  -d '{"title": "Implement API", "description": "Create REST API endpoints",
    "project_id": 1, "priority": "high"}'
```

Get All Tasks

GET /tasks/

Retrieve all tasks in the system.

Response:

```
[
  {
    "id": 1,
    "title": "Task 1",
    "description": "Description 1",
    "status": "pending",
    "priority": "medium",
    "project_id": 1,
    "assigned_to": null,
    "created_at": "2024-01-01T00:00:00",
    "updated_at": "2024-01-01T00:00:00"
  }
]
```

Get Task by ID

GET /tasks/{task_id}

Retrieve a specific task by its ID.

Parameters:

- `task_id` (integer): The ID of the task

Documents

Create Document

POST /documents/

Create a new document within a project.

Request Body:

```
{
  "title": "string",
  "content": "string",
  "project_id": "integer"
}
```

Response:

```
{
  "id": 1,
  "title": "Document Title",
  "content": "Document content",
  "file_path": null,
  "project_id": 1,
  "created_at": "2024-01-01T00:00:00",
  "updated_at": "2024-01-01T00:00:00"
}
```

Example:

```
curl -X POST "http://localhost:8000/documents/" \
  -H "Content-Type: application/json" \
  -d '{"title": "API Specification", "content": "This document describes the API...", "project_id": 1}'
```

Get All Documents

GET /documents/

Retrieve all documents in the system.

Response:

```
[
  {
    "id": 1,
    "title": "Document 1",
    "content": "Content 1",
    "file_path": null,
    "project_id": 1,
    "created_at": "2024-01-01T00:00:00",
    "updated_at": "2024-01-01T00:00:00"
  }
]
```

Agents

Create Agent

POST /agents/

Create a new AI agent in the system.

Request Body:

```
{
  "name": "string",
  "role": "string",
  "description": "string (optional)",
  "config": "string (optional, JSON configuration)"
}
```

Response:

```
{
  "id": 1,
  "name": "Agent Name",
  "role": "Developer",
  "description": "Agent description",
  "status": "active",
  "config": null,
  "created_at": "2024-01-01T00:00:00",
  "updated_at": "2024-01-01T00:00:00"
}
```

Example:

```
curl -X POST "http://localhost:8000/agents/" \
  -H "Content-Type: application/json" \
  -d '{"name": "Code Reviewer", "role": "QA Engineer", "description": "Automated code review agent"}'
```

Get All Agents

GET /agents/

Retrieve all agents in the system.

Response:

```
[
  {
    "id": 1,
    "name": "Agent 1",
    "role": "Developer",
    "description": "Description 1",
    "status": "active",
    "config": null,
    "created_at": "2024-01-01T00:00:00",
    "updated_at": "2024-01-01T00:00:00"
  }
]
```

Health & Status

Health Check

GET /health

Check the health status of the API.

Response:

```
{
  "status": "healthy",
  "service": "SDLC API"
}
```

Root Endpoint

GET /

Get basic information about the API.

Response:

```
{
  "message": "Enhanced Agentic AI SDLC System API",
  "version": "1.0.0"
}
```

Error Handling

Error Response Format

```
{
  "detail": "Error message description"
}
```

Common HTTP Status Codes

Status Code	Description
200	Success
201	Created
400	Bad Request
404	Not Found
422	Validation Error
500	Internal Server Error

Example Error Responses

404 Not Found:

```
{
  "detail": "Project not found"
}
```

422 Validation Error:

```
{
  "detail": [
    {
      "loc": ["body", "name"],
      "msg": "field required",
      "type": "value_error.missing"
    }
  ]
}
```

Development Tools

Interactive API Documentation

Visit `http://localhost:8000/docs` for Swagger UI documentation with interactive testing capabilities.

Alternative Documentation

Visit `http://localhost:8000/redoc` for ReDoc-style documentation.

Request/Response Examples

Complete Project Workflow

```
# 1. Create a project
PROJECT_RESPONSE=$(curl -s -X POST "http://localhost:8000/projects/" \
  -H "Content-Type: application/json" \
  -d '{"name": "Sample Project", "description": "A sample project"}')

PROJECT_ID=$(echo $PROJECT_RESPONSE | jq -r '.id')

# 2. Create a task for the project
curl -X POST "http://localhost:8000/tasks/" \
  -H "Content-Type: application/json" \
  -d '{"title": "Sample Task", "description": "A sample task", "project_id": $PROJECT_ID}'

# 3. Create a document for the project
curl -X POST "http://localhost:8000/documents/" \
  -H "Content-Type: application/json" \
  -d '{"title": "Project Documentation", "content": "This is the project documentation", "project_id": $PROJECT_ID}'

# 4. Create an agent
curl -X POST "http://localhost:8000/agents/" \
  -H "Content-Type: application/json" \
  -d '{"name": "Project Assistant", "role": "Project Manager", "description": "Helps manage the project"}'
```

Future Endpoints

The following endpoints are planned for future releases:

- **PUT** `/projects/{project_id}` - Update project
- **DELETE** `/projects/{project_id}` - Delete project
- **PUT** `/tasks/{task_id}` - Update task
- **DELETE** `/tasks/{task_id}` - Delete task
- **POST** `/tasks/{task_id}/assign` - Assign task to agent
- **POST** `/agents/{agent_id}/execute` - Execute agent task
- **GET** `/projects/{project_id}/tasks` - Get tasks for project
- **GET** `/projects/{project_id}/documents` - Get documents for project
- **POST** `/documents/upload` - Upload file document

- **GET** `/search` - Search across all content

Rate Limiting

Currently, no rate limiting is implemented. In production, the following limits will apply:

- 100 requests per minute per IP
- 1000 requests per hour per authenticated user
- Special limits for file upload endpoints

Security Considerations

Current Security Measures

- CORS configuration
- Input validation with Pydantic
- SQL injection prevention with SQLAlchemy

Planned Security Features

- JWT authentication
- Role-based access control
- API key management
- Request rate limiting
- Input sanitization
- Audit logging

For more detailed information, visit the interactive documentation at `http://localhost:8000/docs` when the API is running.